UNIVERSITY OF EASTERN PANGASINAN
COLLEGE OF INFORMATION TECHNOLOGY
BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

Technical Defense Guide & Speaking Script

Comprehensive 15-Minute Oral Defense Script & Examiner Demonstration Runbook
Mission 10: Enterprise Cloud Infrastructure

———————————————————————————

Student / Candidate: **Carlo Nimer**

Year & Section: Fourth Year – BSIT-4J

Course / Mission: **CCM101 (Cloud Computing) — Final Requirement (Mission 10)**

Instructor / Evaluator: Prof. Jenkielyn C. Torres

Academic Year: 2026–2027

Date of Submission: October 18, 2026

Examiner Runbook & Candidate Presentation Strategy

This technical defense guide provides the exact verbal script, screen presentation flow, live command sequences, expected visual feedback, and instructor Q&A prep for the 15+ minute recorded technical defense of the Enterprise Cloud Infrastructure final project.

The defense is structured into 13 logical, progressive stages designed to demonstrate engineering competency across virtualization, networking, container orchestration, application observability, and disaster recovery.

Phase	Defense Subject Matter	Timeline
Stages 1 – 3	Host, Hypervisor & SSH Management Flow	00:00 – 03:30 (3.5 mins)
Stages 4 – 5	Guest Hardware Audit & Docker Compose Topology	03:30 – 06:30 (3.0 mins)
Stages 6 – 9	Multi-Tier Container Inspection (Nginx, Flask, DB, Volume)	06:30 – 10:30 (4.0 mins)
Stages 10 – 12	Security (UFW), Telemetry (/health) & Disaster Recovery	10:30 – 14:00 (3.5 mins)
Stage 13	Limitations, Future Enhancements & Defense Closing	14:00 – 15:30 (1.5 mins)

Stage 1: Introduction, Candidate Profile & Project Objectives (1.5 minutes (00:00 – 01:30))

Stage Parameter	Runbook Directive
Target Duration:	1.5 minutes (00:00 – 01:30)
Display on Screen:	Display official architecture diagram (architecture-diagram.png) in full-screen viewer.
Evidence to Capture:	Screenshot of the opening title slide and full-screen architecture diagram.

Verbal Speaking Script (Candidate Voice):

"Good day, Professor Torres and members of the evaluation committee. My name is Carlo Nimer from section BSIT-4J. Today, I am presenting my final requirement in CCM101 Cloud Computing, entitled Enterprise Cloud Infrastructure. The core objective of this project is to architect, deploy, harden, and validate a sovereign, automated, multi-tier private cloud environment on a dedicated virtualized host. Throughout this demonstration, I will prove that every layer of this architecture—from Windows 11 host virtualization and OpenSSH remote administration, to multi-tier Docker Compose microservices, UFW perimeter firewall hardening, active /health telemetry, and automated disaster recovery—is fully implemented, verifiable, and operational."

Expected Live Output Verification:

Clear, calm introductory delivery while presenting the clean architecture diagram.

Instructor Q&A Preparation:

Q: Question: "Why did you choose a private cloud lab instead of deploying directly to AWS or Azure?"

A: Model Answer: "Deploying this sovereign private cloud on localized virtualization demonstrates full-stack architectural mastery—from hypervisor management and kernel networking to Linux server hardening. In public hyperscalers, underlying virtualization and storage controllers are abstracted away. Building it locally demonstrates mastery of both foundational infrastructure and cloud orchestration."

Stage 2: Host Computer & Hypervisor Infrastructure Audit (1.0 minute (01:30 – 02:30))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (01:30 – 02:30)
Display on Screen:	Windows Terminal / PowerShell displaying system info.
Evidence to Capture:	Screenshot of PowerShell host audit command output.

Commands to Execute Live:

```
[WINDOWS POWERSHELL]
Get-ComputerInfo | Select-Object WindowsProductName, OsArchitecture
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors
Get-CimInstance Win32_ComputerSystem | Select-Object
@{N="RAM_GB";E={[math]::Round($_.TotalPhysicalMemory/1GB,2)}}
& 'C:\Program Files\Oracle\VirtualBox\VBoxManage.exe' --version
```

Verbal Speaking Script (Candidate Voice):

"Before inspecting the guest machine, I will verify the physical host environment. I am running Windows 11 64-bit on a Lenovo LOQ workstation powered by a multi-core processor with 16 Gigabytes of physical RAM. Our hypervisor is Oracle VM VirtualBox version 7, which manages our hardware-assisted virtualization."

Expected Live Output Verification:

PowerShell returns Windows 11, physical CPU core counts, total physical memory in GB, and VirtualBox version 7.x.

Instructor Q&A Preparation:

Q: Question: "How does physical RAM capacity affect virtual machine stability?"

A: Model Answer: "Allocating 4 GB to the VM from a 16 GB physical host ensures the guest has dedicated physical pages without forcing the host operating system into disk swapping, preventing memory ballooning and performance degradation."

Stage 3: PowerShell-to-Ubuntu Remote SSH Administration (1.0 minute (02:30 – 03:30))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (02:30 – 03:30)
Display on Screen:	PowerShell terminal executing SSH login.
Evidence to Capture:	Screenshot showing successful SSH login from Windows PowerShell into Ubuntu Server.

Commands to Execute Live:

```
[WINDOWS POWERSHELL]
ssh carlo@192.168.100.52
```

Verbal Speaking Script (Candidate Voice):

"Rather than managing our server through a graphical console, we enforce modern DevOps out-of-band management by establishing an encrypted OpenSSH session from Windows PowerShell directly into the

Ubuntu Server guest at IP 192.168.100.52. Notice that when I enter my password, characters are masked by UNIX design to prevent shoulder surfing. We are now authenticated inside the Linux server."

Expected Live Output Verification:

Successful authentication prompt followed by the Ubuntu 24.04 LTS motd banner and prompt: carlo@ubuntu-docker:~$

Instructor Q&A Preparation:

Q: Question: "What network mode allows PowerShell to connect directly to the VM's 192.168.x.x IP?"

A: Model Answer: "VirtualBox Bridged Networking mode attaches the VM's virtual NIC directly to the host's physical Wi-Fi adapter, making the VM a peer device on the local physical network with its own DHCP-leased IP."

Stage 4: Guest Virtual Hardware & Resource Verification (1.5 minutes (03:30 – 05:00))

Stage Parameter	Runbook Directive
Target Duration:	1.5 minutes (03:30 – 05:00)
Display on Screen:	Ubuntu SSH session running resource inspection commands.
Evidence to Capture:	Screenshot of hostnamectl, free -h, and df -h output.

Commands to Execute Live:

```
[UBUNTU SSH]
hostnamectl
nproc
free -h
swapon --show
df -h /
```

Verbal Speaking Script (Candidate Voice):

"Now inside Ubuntu Server, we verify our allocated virtual hardware. hostnamectl proves we are running a dedicated headless Ubuntu 24.04 LTS instance with the oracle hypervisor chassis. Running nproc confirms exactly 2 virtual CPUs. free -h shows approximately 3.8 GiB of available RAM, supported by an active 2.8 GiB swap partition on virtual disk. Finally, df -h proves our root ext4 filesystem has over 50% free storage space."

Expected Live Output Verification:

Static hostname: ubuntu-docker, 2 CPUs, 3.8Gi Total RAM, 2.8Gi Swap, root filesystem mounted on / with ample free space.

Instructor Q&A Preparation:

Q: Question: "Why does free -h display 3.8 GiB instead of exactly 4.0 GiB?"

A: Model Answer: "The Linux kernel reserves a small memory region (approximately 150-200 MB) during early boot for the kernel code segment, page tables, and hardware I/O memory maps, leaving ~3.8 GiB usable for userland processes."

Stage 5: Docker Compose & Multi-Tier Topology Validation (1.5 minutes (05:00 – 06:30))

Stage Parameter	Runbook Directive
Target Duration:	1.5 minutes (05:00 – 06:30)
Display on Screen:	Project directory in terminal displaying compose status.
Evidence to Capture:	Screenshot of docker compose ps displaying all 3 containers healthy.

Commands to Execute Live:

```
[UBUNTU SSH]
cd /home/carlo/enterprise-cloud
docker compose config -q && echo 'YAML Specification: VALID'
docker compose ps
docker network ls
```

Verbal Speaking Script (Candidate Voice):

"Navigating to /home/carlo/enterprise-cloud, we inspect our container orchestration. Running docker compose config -q validates our declarative Infrastructure-as-Code specification. docker compose ps shows our three microservices: nginx listening on 8088, app running Flask on internal port 5000, and db running PostgreSQL on internal port 5432. docker network ls confirms our dual bridge network topology."

Expected Live Output Verification:

YAML Specification: VALID. Three services reported as 'Up' with green health status.

Instructor Q&A Preparation:

Q: Question: "What is the benefit of defining services in docker-compose.yml over running docker run manually?"

A: Model Answer: "Docker Compose provides declarative, version-controlled Infrastructure-as-Code. It defines network boundaries, environment secrets, persistent volume bindings, and automated healthchecks in a single reproducible manifest."

Stage 6: Presentation Tier: Nginx Reverse Proxy (:8088) (1.0 minute (06:30 – 07:30))

Stage Parameter	Runbook Directive

Target Duration:	1.0 minute (06:30 – 07:30)
Display on Screen:	Browser loading http://192.168.100.52:8088 and terminal curl check.
Evidence to Capture:	Screenshot of browser loading the CloudNova web portal alongside Nginx access logs.

Commands to Execute Live:

```
[UBUNTU SSH]
curl -i http://127.0.0.1:8088/
docker compose logs --tail=10 nginx
```

Verbal Speaking Script (Candidate Voice):

"Here is our presentation tier. Nginx acts as our front-line reverse proxy. It terminates client connections on port 8088, protects backend application servers from direct exposure, and forwards valid HTTP traffic to the Flask container. In the browser, our modern corporate portal loads instantly, demonstrating sub-second response times."

Expected Live Output Verification:

HTTP/1.1 200 OK header followed by corporate portal HTML markup; Nginx access logs recording the request with status 200.

Instructor Q&A Preparation:

Q: Question: "Why run Nginx as a reverse proxy instead of exposing Flask directly on port 8088?"

A: Model Answer: "Nginx is highly optimized for asynchronous network I/O, TLS termination, header sanitization, and request rate limiting. Exposing Gunicorn/Flask directly creates security and denial-of-service risks."

Stage 7: Application & Database Tiers: Flask + PostgreSQL 16 (1.5 minutes (07:30 – 09:00))

Stage Parameter	Runbook Directive
Target Duration:	1.5 minutes (07:30 – 09:00)
Display on Screen:	Web application interactive form and database log inspection.
Evidence to Capture:	Screenshot of SQL query verifying data insertion from the web portal.

Commands to Execute Live:

```
[UBUNTU SSH]
docker compose exec -T db psql -U postgres -d enterprise_cloud_db -c 'SELECT count(*)
FROM workloads;'
```

Verbal Speaking Script (Candidate Voice):

"Behind Nginx sits our application tier: a Python Flask microservice executed by Gunicorn with 2 workers and 2 threads. The app communicates with PostgreSQL 16 over the isolated internal backend network. As you see on screen, submitting a workload in the web portal writes a transactional row directly into PostgreSQL, confirmed by querying the database container."

Expected Live Output Verification:

SQL query returning current workload count; proof that web form submissions commit to PostgreSQL.

Instructor Q&A Preparation:

Q: Question: "How does the Flask container know how to resolve the database host 'db'?"

A: Model Answer: "Docker Compose provides built-in DNS service discovery on user-defined bridge networks. The service name 'db' is automatically resolved to the database container's internal IP address."

Stage 8: State Persistence: Named Docker Volumes (1.0 minute (09:00 – 10:00))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (09:00 – 10:00)
Display on Screen:	Terminal displaying volume inspect and restart test.
Evidence to Capture:	Screenshot of docker volume inspect output.

Commands to Execute Live:

```
[UBUNTU SSH]
docker volume ls
docker volume inspect enterprise-cloud_postgres_data
```

Verbal Speaking Script (Candidate Voice):

"In enterprise architectures, compute must be stateless while data must be persistent. We achieve this by binding PostgreSQL's data directory (/var/lib/postgresql/data) to a named Docker volume called postgres_data. Even if the container is destroyed or recreated, the physical data on disk remains 100% intact."

Expected Live Output Verification:

Docker volume metadata showing driver: local and mountpoint path inside /var/lib/docker/volumes/.

Instructor Q&A Preparation:

Q: Question: "What happens to database records if you execute 'docker compose down'?"

A: Model Answer: "The containers and virtual networks are stopped and removed, but the named volume remains completely untouched on disk. When 'docker compose up -d' is executed, the new container mounts the existing volume with zero data loss."

Stage 9: Perimeter Defense & UFW Firewall Policy (1.0 minute (10:00 – 11:00))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (10:00 – 11:00)
Display on Screen:	Terminal displaying UFW firewall status and socket listening ports.
Evidence to Capture:	Screenshot of sudo ufw status verbose displaying active firewall rules.

Commands to Execute Live:

```
[UBUNTU SSH]
sudo ufw status verbose
sudo ss -tulpn | grep -E '8088|22|5000|5432'
```

Verbal Speaking Script (Candidate Voice):

"Perimeter security is enforced using Uncomplicated Firewall (UFW). Our firewall policy is configured with Default Deny Inbound and Default Allow Outbound. Only ports 22 for SSH and 8088 for Nginx are permitted. Notice from ss -tulpn that neither port 5000 nor port 5432 is exposed to the physical network. The database is strictly isolated."

Expected Live Output Verification:

Status: active. Default: deny (incoming), allow (outgoing). Rules allow 22/tcp and 8088/tcp.

Instructor Q&A Preparation:

Q: Question: "If UFW blocks incoming traffic, how do Docker containers communicate with each other?"

A: Model Answer: "Docker container-to-container communication occurs across internal virtual bridge interfaces (br-xxxx), bypassing the host's physical network interface where UFW filters external ingress."

Stage 10: Application Observability & Active /health Telemetry (1.0 minute (11:00 – 12:00))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (11:00 – 12:00)
Display on Screen:	Terminal executing curl to health probe.

Evidence to Capture:	Screenshot of HTTP 200 OK health check response.

Commands to Execute Live:

```
[UBUNTU SSH]
curl -i -sS http://127.0.0.1:8088/health
```

Verbal Speaking Script (Candidate Voice):

"For enterprise observability, we implemented an active telemetry endpoint at /health. When queried, Flask executes an active live ping against PostgreSQL. If the database socket responds, it returns HTTP 200 OK with JSON payload status healthy and database connected. This probe executes in under 15 milliseconds."

Expected Live Output Verification:

HTTP/1.1 200 OK
{"database":"connected","status":"healthy"}

Instructor Q&A Preparation:

Q: Question: "Why is an application-level health probe superior to just checking if the container is running?"

A: Model Answer: "A container process can be running while the internal application thread is deadlocked or unable to connect to the database. An active /health probe verifies true end-to-end functional health."

Stage 11: Autonomous Backup Operations & Cron Scheduling (1.0 minute (12:00 – 13:00))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (12:00 – 13:00)
Display on Screen:	Terminal displaying crontab, backup script permissions, and backups directory.
Evidence to Capture:	Screenshot of crontab -l and ls -lah backups/ output.

Commands to Execute Live:

```
[UBUNTU SSH]
crontab -l
ls -lah /home/carlo/enterprise-cloud/backups/
sudo systemctl status cron --no-pager | head -n 8
```

Verbal Speaking Script (Candidate Voice):

"Our disaster recovery strategy includes autonomous daily backups. crontab -l reveals our scheduled job: at 02:00 AM daily, cron executes automation-script.sh. The script performs a non-blocking pg_dump, streams the SQL through gzip compression, applies umask 077 for security, and rotates archives older than 7 days."

Expected Live Output Verification:

Cron job definition 0 2 * * * /home/carlo/enterprise-cloud/automation-script.sh; backup archives listed with -rw------- permissions.

Instructor Q&A Preparation:

Q: Question: "Why is umask 077 used in the backup script?"

A: Model Answer: "umask 077 ensures that generated backup files are readable and writable solely by user 'carlo', preventing any other local user from reading confidential database dumps."

Stage 12: Live Non-Destructive Disaster Recovery Verification (1.5 minutes (13:00 – 14:30))

Stage Parameter	Runbook Directive
Target Duration:	1.5 minutes (13:00 – 14:30)
Display on Screen:	Terminal executing backup validation and test database restoration.
Evidence to Capture:	Screenshot of successful non-destructive database restoration output.

Commands to Execute Live:

```
[UBUNTU SSH]
LATEST=$(ls -t /home/carlo/enterprise-cloud/backups/cloudapp_*.sql.gz | head -1)
gzip -t "$LATEST" && echo "Archive Integrity: PASS"
docker compose exec -T db psql -U postgres -c "DROP DATABASE IF EXISTS
cloudapp_restore_test;"
docker compose exec -T db psql -U postgres -c "CREATE DATABASE cloudapp_restore_test;"
gunzip -c "$LATEST" | docker compose exec -T db psql -U postgres -d
cloudapp_restore_test
docker compose exec -T db psql -U postgres -d cloudapp_restore_test -c "SELECT count(*)
FROM workloads;"
docker compose exec -T db psql -U postgres -c "DROP DATABASE cloudapp_restore_test;"
echo "Disaster recovery verified."
```

Verbal Speaking Script (Candidate Voice):

"Now, I will prove that our disaster recovery procedure actually works. We select the latest backup file and run gzip -t to verify CRC integrity. Next, we create a temporary test database, stream the restored dump into it, count the records to prove data integrity, and cleanly drop the test database. This verifies complete disaster recovery capability in under 45 seconds without touching production."

Expected Live Output Verification:

Archive Integrity: PASS, CREATE DATABASE, table schema loaded, record count displayed, Disaster recovery verified.

Instructor Q&A Preparation:

Q: Question: "What is the difference between RPO and RTO in this project?"

A: Model Answer: "Recovery Point Objective (RPO) is 24 hours, determined by the daily 2:00 AM backup frequency. Recovery Time Objective (RTO) is under 2 minutes, the time required to decompress and stream the SQL dump into a database instance."

Stage 13: Technical Limitations, Future Enhancements & Defense Closing (1.0 minute (14:30 – 15:30))

Stage Parameter	Runbook Directive
Target Duration:	1.0 minute (14:30 – 15:30)
Display on Screen:	Return to architecture diagram slide.
Evidence to Capture:	Final recording timestamp and slide presentation screenshot.

Verbal Speaking Script (Candidate Voice):

"To conclude, Mission 10 has achieved all core objectives: containerized microservices orchestration, state persistence, host perimeter hardening, observability, and verifiable automated backups. Acknowledging current limitations: backups are stored locally on the VM virtual disk, and ingress is HTTP. In production, we would synchronize backups to off-site cloud storage (such as AWS S3), enable TLS 1.3 certificates via Let's Encrypt, and deploy Prometheus sidecars. Thank you for your time, and I am now ready for any additional questions."

Expected Live Output Verification:

Polished, confident closing statement delivered within the 15-minute timeframe.

Instructor Q&A Preparation:

Q: Question: "What is the single most important lesson you learned building this infrastructure?"

A: Model Answer: "The critical lesson was understanding separation of concerns across infrastructure layers: isolating network boundaries, decoupling state into volumes, and validating disaster recovery through active testing rather than passive assumption."